

Creating a Custom Dataset for Your Own Files in PyTorch

Lab Objective:

The objective of this lab is to help students understand how to load their own data into PyTorch by creating a custom dataset class. Students will learn how to organize files, read data from folders or CSV files, apply transformations, and use DataLoader for batching and shuffling.

Learning Outcomes:

By the end of this lab, students will be able to:

- understand the need for custom datasets in PyTorch
- create a dataset class by inheriting from Dataset
- implement `__init__()`, `__len__()`, and `__getitem__()` methods
- load image data from their own folders
- apply preprocessing and transformations
- use DataLoader to prepare data for model training

Introduction:

In real-world deep learning tasks, datasets are often not available in ready-made PyTorch format. Instead, data may be stored in folders, subfolders, or CSV files. PyTorch provides the Dataset and DataLoader classes to make data handling flexible and efficient.

A custom dataset allows us to define how data is read and returned to the model. This is especially useful when working with personal datasets such as images, text files, audio files, or tabular data.

Pre-Lab Requirements:

Students should have:

- basic knowledge of Python classes
- basic understanding of PyTorch
- PyTorch, torchvision, pandas, and PIL installed
- a dataset stored in folders on their computer

Tools and Libraries Required:

- Python

- PyTorch
- torchvision
- pandas
- PIL (Python Imaging Library)
- matplotlib

Dataset Folder Structure:

Suppose we have an image dataset stored in the following structure:

```
dataset/  
  cats/  
    cat1.jpg  
    cat2.jpg  
    cat3.jpg  
  dogs/  
    dog1.jpg  
    dog2.jpg  
    dog3.jpg
```

Here:

- cats and dogs are class labels
- each folder contains images belonging to that class

Custom Dataset:

A custom dataset class must define three main methods:

1. `__init__()`

This method is used to initialize the dataset, load file paths, labels, and transformations.

2. `__len__()`

This method returns the total number of samples in the dataset.

3. `__getitem__()`

This method retrieves one sample at a given index and returns the input data with its label.

Basic Workflow:

The general workflow for creating a custom dataset is:

1. import required libraries
2. define dataset class
3. collect file paths and labels
4. read files in `__getitem__()`
5. apply transformations
6. return data and label
7. use DataLoader for batching

Creating a Custom Dataset for Image Files

Step 01:

Create a dummy dataset. Download the images of cats and dogs from google (20 each). And save them according to the above-mentioned directory.

Step 02:

Import the require libraries.

```
import os
from PIL import Image
import torch
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
```

Step 03:

Create custom dataset class

```
class CustomImageDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.image_paths = []
        self.labels = []
        self.class_names = os.listdir(root_dir)

        for label, class_name in enumerate(self.class_names):
            class_folder = os.path.join(root_dir, class_name)
            for file_name in os.listdir(class_folder):
                file_path = os.path.join(class_folder, file_name)
                self.image_paths.append(file_path)
                self.labels.append(label)

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        image = Image.open(self.image_paths[idx]).convert("RGB")
        label = self.labels[idx]

        if self.transform:
            image = self.transform(image)

        return image, label
```

Step 04:

Define transforms.

```
transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor()
])
```

✓ 0.0s

Step 05:

Create dataset object and dataloader.

```
dataset = CustomImageDataset(root_dir="dataset", transform=transform)
```

✓ 0.0s

```
dataloader = DataLoader(dataset, batch_size=5, shuffle=True)
```

✓ 0.0s

Step 06:

Check results.

```
for images, labels in dataloader:
    print("Image batch shape:", images.shape)
    print("Labels:", labels)
    break
```

✓ 0.0s

Image batch shape: torch.Size([5, 3, 128, 128])
Labels: tensor([1, 0, 0, 1, 0])

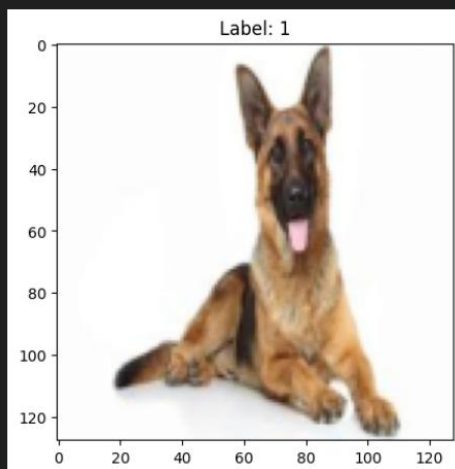
Step 07:

Display image from the dataset.

```
import matplotlib.pyplot as plt

for images, labels in dataloader:
    image = images[0].permute(1, 2, 0)
    plt.imshow(image)
    plt.title(f"Label: {labels[0].item()}")
    plt.show()
    break
```

✓ 2.3s



Creating a Custom Dataset for Image file paths and labels stored in CSV

Sometimes file names and labels are stored in a CSV file.

filename	Label
Cat1.jpg	0
Cat2.jpg	0
Dog1.jpg	1
Dog2.jpg	1

Step 01:

Create above like csv for your custom dataset.

Step 02:

Create custom dataset class and check results.

```
import pandas as pd

class CSVImageDataset(Dataset):
    def __init__(self, csv_file, root_dir, transform=None):
        self.data = pd.read_csv(csv_file)
        self.root_dir = root_dir
        self.transform = transform

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        img_name = self.data.iloc[idx, 0]
        label = self.data.iloc[idx, 1]
        img_path = os.path.join(self.root_dir, img_name)

        image = Image.open(img_path).convert("RGB")

        if self.transform:
            image = self.transform(image)

        return image, label
```

```
from torch.utils.data import DataLoader

dataset = CSVImageDataset(
    csv_file="dataset.csv",
    root_dir="dataset2",
    transform=transform
)

dataloader = DataLoader(dataset, batch_size=4, shuffle=True)
```

6] ✓ 0.0s

```
import matplotlib.pyplot as plt

image, label = dataset[0]

# Convert tensor to image format
image = image.permute(1, 2, 0)

plt.imshow(image)
plt.title(f"Label: {label}")
plt.axis('off')
plt.show()
```

✓ 0.4s



Lab Task:

1. Practice all above examples.
2. Download the Intel Image Classification Dataset from Kaggle. URL: <https://www.kaggle.com/datasets/blourdhuraju/intel-image-classification-dataset>
3. Write down the custom dataset class using image files and folders. Check results and display images.
4. Now alternatively, write down a python script that would create a csv file for the dataset directory and then create custom dataset class for csv and image paths.
5. Check results and display them.
6. Include all the relevant screenshots in your lab manual.